



Reu Reu Vending Machine Simulator By Reuben Thomas

My project is a text-based vending machine.

Simulate a vending machine experience in Java

- Console-based menu
- Balance tracking & penalties
- Objective: Buy 3 snacks before running out of money

```
70      Snack selected = store.getSnack(index);
71
72  }
73  }
74  }
```

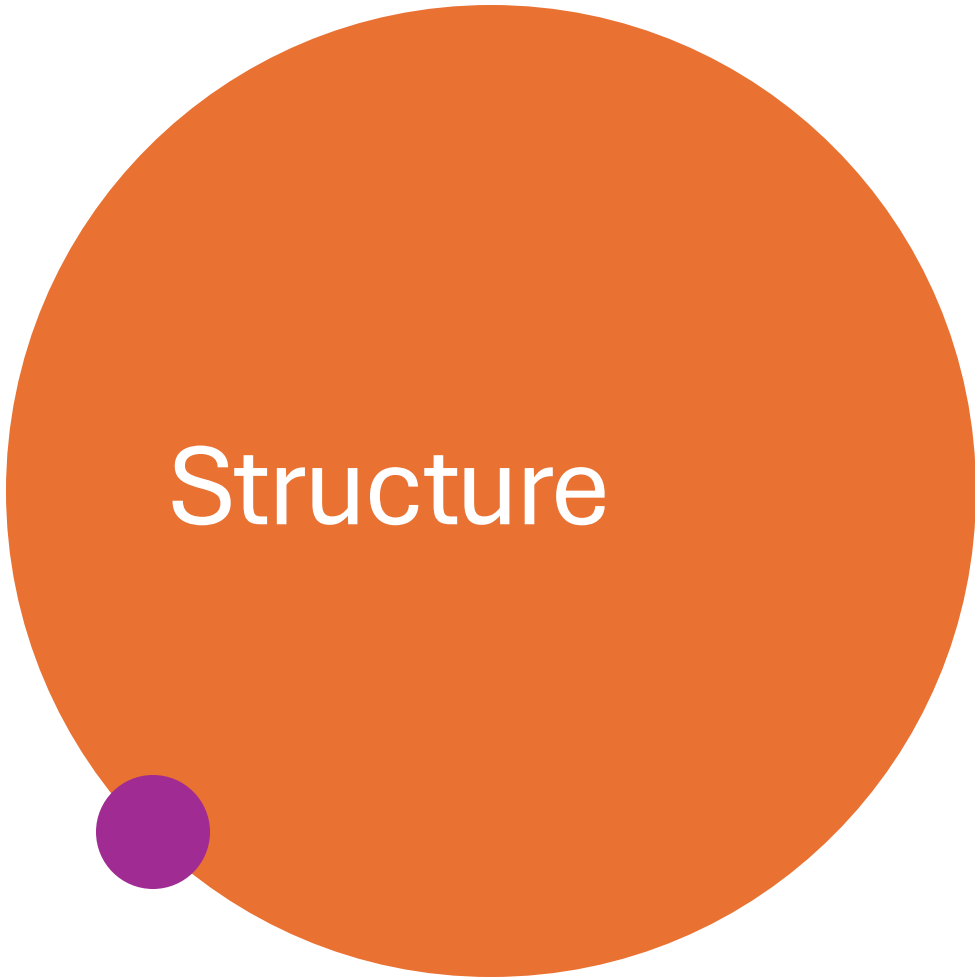
Problems Javadoc Declaration Search Console X Eclipse IDE for Java Developers 2025-09 Release

Main (1) [Java Application] C:\Users\muffi\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v202507

```
===> Welcome to Reuben Vending machine! <===
-----
      balance: $8.12
-----

Objective: Buy 3 snacks before running out of money

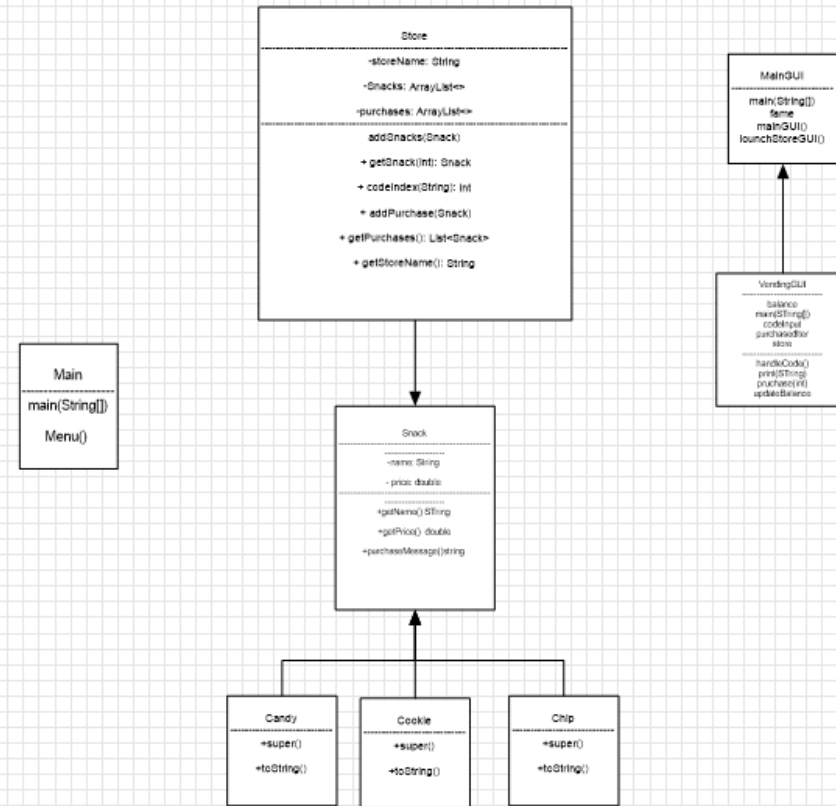
Click Enter to see the store menu...
```



Structure

- Main class: Handles game loop, balance, purchases
- Store class: Manages snacks, menu, purchase history
- Snack (base class)
Candy, Cookie, Chip (subclasses with custom messages)
- GUI classes: VendingGUI, MainGUI

UML Diagram



Inheritance and polymorphism

```
Chip.java ×
1 package VendingMachine;
2
3 public class Chip extends Snack {
4
5     /**
6      * Chip snack type.
7      */
8
9     public Chip(String name, double
10         super(name, price);
11     }
12     @Override
13     public String toString() {
14         return "[Chip] " + super.to
15     }
16     @Override
17     public String purchaseMessage()
18         return "Crunchy choice! En
19     }
20
21 }

Candy.java ×
1 package VendingMachine;
2
3 public class Candy extends Snack {
4
5     /**
6      * Candy snack type.
7      */
8
9     public Candy(String name, double
10         super(name, price);
11     }
12     @Override
13     public String toString() {
14         return "[Candy] " + super.t
15     }
16     @Override
17     public String purchaseMessage()
18         return "Sugar rush! Enjoy
19     }
20 }
21 }

Cookie.java ×
1 package VendingMachine;
2
3 public class Cookie extends Snack {
4
5     /**
6      * Cookie snack type.
7      */
8
9     public Cookie(String name, double price) {
10         super(name, price);
11     }
12     @Override
13     public String toString() {
14         return "[Cookie] " + super.toString();
15     }
16     @Override
17     public String purchaseMessage() {
18         return "Sweet treat! Enjoy your cookie!";
19     }
20 }
```

- Inheritance: my base class (Snack) I extends 3 subclass to it. "
Snack(super(name,price));"
- Polymorphism: my method name (purchaseMessage) is different depending on the object type

substantive collections

```
8  */
9
10
11 public class Store {
12
13     private String storeName;
14     private ArrayList<Snack> Snacks;
15     private ArrayList<Snack> purchases;
16 }
```

- My ArrayList<snacks>Snacks hold all the available snacks in the vending machine. This make it a substantive collection because it store multiple objects of different subclass
- ArrayList<snacks>purchase records the history, print at end.

```

4      System.out.println("");
5      System.out.println("    Click Enter to see the store menu...");
6      input.nextLine();
7      // store.showMenu();
8
9      Menu();
10     try {
11
12         System.out.println("\nEnter item code or X to exit: ");
13         String code = input.nextLine();
14
15         //String code = null;
16         if(code.equalsIgnoreCase("X")) {
17             System.out.println("Exiting.....");
18             break;
19         }
20
21         int index = store.codeIndex(code);
22         Snack selected = store.getSnack(index);
23

```

```

}
} catch (IllegalArgumentException e) {
    System.out.println("Error: " + e.getMessage());
} catch (RuntimeException e) {
    System.out.println("Error: " + e.getMessage());
    running = false;
}
}

```

Exception handling

- Keeps program running
- Improves reliability
- Provides clear error messages